

CASO G – Calculadora de Planilla de Sueldos

Ejemplo completamente resuelto · 2025

■ **¿Cómo usar este documento?** Lee primero el enunciado e intenta resolverlo por tu cuenta. Luego estudia la solución bloque por bloque. Puedes copiar y pegar el código en Python para ejecutarlo.

1. Enunciado del Problema

Una pequeña empresa de construcción necesita calcular la planilla de sueldos mensual de sus trabajadores. El pago se calcula según las horas trabajadas y una tarifa por hora individual para cada trabajador.

Datos de entrada

- Número de trabajadores (mínimo 2).
- Para cada trabajador: nombre, horas trabajadas en el mes y tarifa por hora (\$).

Reglas de cálculo

- Las primeras 45 horas mensuales se pagan a tarifa normal.
- Las horas que superen las 45 se pagan con un recargo del 50 % ($\text{tarifa} \times 1,5$).
- $\text{Pago total} = \text{pago_base} + \text{pago_extra}$

Resultados requeridos

- Tabla con: nombre, horas normales, horas extra, pago base, pago extra y pago total.
- Total de la planilla (suma de todos los pagos).
- Trabajador con el mayor sueldo y trabajador con el menor sueldo.

■ **Restricciones:** Use listas paralelas (una lista por campo). Defina una función para cada bloque de cálculo. Valide que las horas y la tarifa sean valores positivos.

2. Solución Comentada

La solución está dividida en 6 bloques. Cada bloque es una función independiente. Léelos en orden: cada uno se apoya en los anteriores.

Bloque 1 – Constantes del programa

```
"""
Caso 6 - Calculadora de Planilla de Sueldos
Taller de Programacion I
"""

# Constantes: defínelas al inicio, fuera de cualquier función.
# Si las reglas cambian, solo modificas estos valores.
HORAS_BASE = 45 # horas sin recargo
RECARGO_EXTRA = 1.5 # factor para horas extra (50 % más)
SEPARADOR = "=" * 68
```

Las constantes se escriben en MAYÚSCULAS por convención. Al estar al inicio del archivo, cualquier función puede usarlas sin necesidad de recibirlas como parámetros.

Bloque 2 – Función auxiliar de validación

```
def ingresar_numero(mensaje, minimo=0.01):
    """
    Solicita un número decimal mayor que "minimo".
    Repite la pregunta hasta que el dato sea válido.
    Retorna el valor ingresado.
    """
    while True:
        try:
            valor = float(input(mensaje))
            if valor < minimo:
                print(f" Ingrese un valor mayor o igual a {minimo}.")
            else:
                return valor # valor válido: salimos del ciclo
        except ValueError:
            print(" Ingrese un número válido (use punto decimal).")
```

Esta función encapsula la validación. Al usarla en el resto del código evitamos repetir el bloque try/except en cada input(). El parámetro minimo=0.01 tiene valor por defecto, por lo que es opcional.

Bloque 3 – Ingreso de datos con listas paralelas

```
def ingresar_trabajadores():
    """
    Pide los datos de n trabajadores.
    Retorna tres listas paralelas: nombres, horas, tarifas.
    El trabajador i tiene: nombres[i], horas[i], tarifas[i].
    """
    # Pedir cantidad con validación
    while True:
        try:
            n = int(input(" Cantidad de trabajadores (min 2): "))
```

```

        if n >= 2:
            break
        print(" Deben ser al menos 2 trabajadores.")
    except ValueError:
        print(" Ingrese un número entero.")

nombres = [] # lista de nombres
horas = [] # lista de horas trabajadas en el mes
tarifas = [] # lista de tarifas por hora

for i in range(n):
    print(f"\n Trabajador {i+1} de {n}")
    nombre = input(" Nombre : ")
    h = ingresar_numero(" Horas del mes : ", 1)
    t = ingresar_numero(" Tarifa ($/hora) : ", 1)
    nombres.append(nombre)
    horas.append(h)
    tarifas.append(t)

return nombres, horas, tarifas

```

Las tres listas crecen en paralelo dentro del mismo ciclo. La función retorna las tres listas a la vez: nombres, horas, tarifas = ingresar_trabajadores().

Bloque 4 – Cálculo del pago de un trabajador

```

def calcular_pago(horas, tarifa):
    """
    Recibe las horas trabajadas y la tarifa por hora.
    Retorna una tupla: (pago_base, pago_extra, pago_total).
    Esta función NO imprime nada: solo calcula.
    """
    if horas <= HORAS_BASE:
        # Sin horas extra
        horas_normales = horas
        horas_extra = 0
    else:
        # Con horas extra
        horas_normales = HORAS_BASE
        horas_extra = horas - HORAS_BASE

    pago_base = horas_normales * tarifa
    pago_extra = horas_extra * tarifa * RECARGO_EXTRA
    pago_total = pago_base + pago_extra

    return pago_base, pago_extra, pago_total

```

La función retorna una tupla con tres valores. En el programa principal los desempaquetamos en una línea: base, extra, total = calcular_pago(horas[i], tarifas[i]).

Bloque 5 – Buscar el mayor y menor sueldo

```
def encontrar_extremo(nombres, pagos, buscar_maximo):
    """
    Busca el trabajador con mayor o menor sueldo.
    Si buscar_maximo es True --> busca el mayor.
    Si buscar_maximo es False --> busca el menor.
    Retorna (nombre, pago) del trabajador encontrado.
    """
    idx_extremo = 0 # asumimos que el índice 0 es el extremo inicial

    for i in range(1, len(pagos)):
        if buscar_maximo and pagos[i] > pagos[idx_extremo]:
            idx_extremo = i
        if not buscar_maximo and pagos[i] < pagos[idx_extremo]:
            idx_extremo = i

    return nombres[idx_extremo], pagos[idx_extremo]
```

Un parámetro booleano (buscar_maximo) evita escribir dos funciones casi idénticas. El recorrido parte desde el índice 1 porque el 0 ya es el punto de comparación inicial.

Bloque 6 – Mostrar resultados

```
def mostrar_resultados(nombres, horas, pagos_base, pagos_extra, pagos_totales):
    """
    Imprime la tabla completa y el resumen de la planilla.
    Recibe las listas ya calculadas. No recalcula nada.
    """
    print(f"\n{SEPARADOR}")
    print(" PLANILLA DE SUELDOS MENSUAL")
    print(SEPARADOR)

    # Encabezado de la tabla
    print(f" {'NOMBRE':<20} {'H.NORM':>6} {'H.EXT':>6} {'PAGO BASE':>10} {'P.EXTRA':>9}
{'TOTAL':>10}")
    print(" " + "-" * 64)

    # Fila por cada trabajador
    for i in range(len(nombres)):
        h_norm = min(horas[i], HORAS_BASE)
        h_extra = max(0, horas[i] - HORAS_BASE)
        print(f" {nombres[i]:<20} {h_norm:>6.0f} {h_extra:>6.0f} "
              f"{pagos_base[i]:>10,.0f} {pagos_extra[i]:>9,.0f} {pagos_totales[i]:>10,.0f}")

    # Totales
    total_planilla = sum(pagos_totales)
    print(f"\n TOTAL PLANILLA : ${total_planilla:,.0f}")

    # Mayor y menor sueldo
    nombre_max, pago_max = encontrar_extremo(nombres, pagos_totales, True)
    nombre_min, pago_min = encontrar_extremo(nombres, pagos_totales, False)
    print(f" Mayor sueldo : {nombre_max} --> ${pago_max:,.0f}")
```

```
print(f" Menor sueldo : {nombre_min} --> ${pago_min:,.0f}")
print(SEPARADOR)
```

Esta función solo imprime: recibe las listas ya procesadas. Llama a `encontrar_extremo()` dos veces, una con `True` y otra con `False`.

Bloque 7 – Función main (programa principal)

```
def main():
    """
    Función principal: llama a las demás en orden.
    1. Pide datos 2. Calcula pagos 3. Muestra resultados.
    """

    print(SEPARADOR)
    print(" CALCULADORA DE PLANILLA DE SUELDOS")
    print(SEPARADOR)

    # Paso 1: ingresar datos --> tres listas paralelas
    nombres, horas, tarifas = ingresar_trabajadores()

    # Paso 2: calcular el pago de cada trabajador
    pagos_base = []
    pagos_extra = []
    pagos_totales = []

    for i in range(len(nombres)):
        base, extra, total = calcular_pago(horas[i], tarifas[i])
        pagos_base.append(base)
        pagos_extra.append(extra)
        pagos_totales.append(total)

    # Paso 3: mostrar resultados
    mostrar_resultados(nombres, horas, pagos_base, pagos_extra, pagos_totales)

# Punto de entrada del programa
if __name__ == "__main__":
    main()
```

La función `main()` actúa como director: llama a las demás funciones en orden. El cálculo se hace en un ciclo que recorre las listas de entrada y acumula los resultados en tres nuevas listas. La línea `if __name__ == "__main__"` es buena práctica y hace que el programa ejecute solo cuando se corre directamente.

3. Ejemplo de Ejecución y Salida Esperada

Con los siguientes datos de entrada:

Trabajador	Horas mes	Tarifa (\$/h)
Carlos Soto	50	4.500
Ana Pérez	45	5.200
Luis Mora	38	4.800

Salida del programa:

Salida en consola

```
=====
      CALCULADORA DE PLANILLA DE SUELDOS
=====
NOMBRE H.NORM H.EXT PAGO BASE P.EXTRA TOTAL
-----
Carlos Soto 45 5 202.500 33.750 236.250
Ana Perez 45 0 234.000 0 234.000
Luis Mora 38 0 182.400 0 182.400

TOTAL PLANILLA : $652.650
Mayor sueldo : Carlos Soto --> $236.250
Menor sueldo : Luis Mora --> $182.400
=====
```

4. Verificación Manual de los Cálculos

Comprueba los resultados a mano antes de confiar en tu código:

Trabajador	H. normales	H. extra	Pago base	Pago extra	Total
Carlos Soto	45	$50 - 45 = 5$	$45 \times \$4.500 = \202.500	$5 \times \$4.500 \times 1,5 = \33.750	\$236.250
Ana Pérez	45	0	$45 \times \$5.200 = \234.000	\$0	\$234.000
Luis Mora	38	0	$38 \times \$4.800 = \182.400	\$0	\$182.400

■ **Consejo de estudio:** Antes de mirar la solución completa, intenta escribir solo la función `calcular_pago()` — es la más corta y la más importante. Luego agrega las demás una a una y prueba cada función por separado antes de integrar todo en el `{c('main')}`.